

Ingeniería en Computación

ELECTRÓNICA DIGITAL

Teórico 5

Funciones Lógicas Combinacionales en VHDL



Universidad Nacional
de **Río Negro**

Register Transfer Level (RTL)-Niveles de Abstracción

- RTL: Metodología de diseño pensando la manipulación de datos como transferencia entre registros.
- En este nivel, utilizamos los bloques combinacionales que vimos en clases anteriores (sumadores, multiplexores, etc.)

	typical blocks	signal representation	time representation	behavioral description	physical description
transistor	transistor, resistor	voltage	continuous function	differential equation	transistor layout
gate	and, or, xor, flip-flop	logic 0 or 1	propagation delay	Boolean equation	cell layout
RT	adder, mux, register	integer, system state	clock tick	extended FSM	RT level floor plan
processor	processor, memory	abstract data type	event sequence	algorithm in C	IP level floor plan

Tipos Datos Nativos

- *real, time, delay_length* **NO** sintetizables.
- No todas las operaciones sobre estos tipos son sintetizables.

```
STD_LOGIC
STD_LOGIC_VECTOR
BOOLEAN
INTEGER
NATURAL
POSITIVE
REAL
BIT
BIT_VECTOR(Natural)
CHARACTER
STRING(POSITIVE)
TIME
DELAY_LENGTH
```

```
--'U','X','0','1','Z','W','L','H','-'
--Natural Range of STD_LOGIC
--True or False
--32 or 64 bits
--Integers >= 0
--Integers > 0
--Floating-point
--'0','1'
--Array of bits
--7-bit ASCII
--Array of characters
--hr, min, sec, ms, us, ns, ps, fs
--Time >= 0
```

Operadores Nativos

- Tipos nativos (32 bits): **integer**, **natural**
- En general queremos mayor manejo del numero de bits en cada señal: usar **signed** o **unsigned** de *IEEE.numeric_std*

Operator	Description	Data type of operands	Data type of result
a ** b	exponentiation	integer	integer
a * b	multiplication	<i>integer type for constants and array boundaries, not synthesis</i>	
a / b	division		
a + b	addition		
a - b	subtraction		
a & b	concatenation	1-D array, element	1-D array
a = b	equal to	any	boolean
a /= b	not equal to	scalar or 1-D array	boolean
a < b	less than		
a <= b	less than or equal to		
a > b	greater than		
a >= b	greater than or equal to		
not a	negation	boolean, std_logic, std_logic_vector	same as operand
a and b	and		
a or b	or		
a xor b	xor		

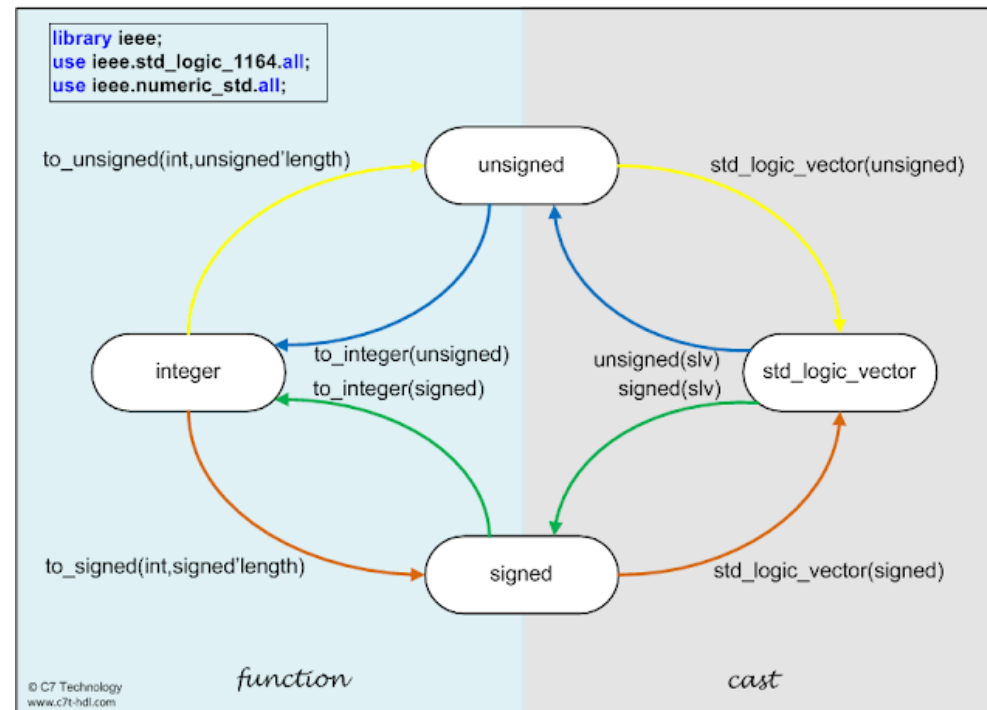
Operadores overloaded *signed* y *unsigned*

- library ieee;
- use ieee.numeric_std.all;
- signal a: unsigned(3 downto 0);

Overloaded operator	Description	Data type of operands	Data type of result
a * b	arithmetic operation	unsigned, natural	unsigned
a + b		signed, integer	signed
a - b			
a = b	relational operation		
a /= b			
a < b		unsigned, natural	boolean
a <= b		signed, integer	boolean
a > b			
a >= b			

Casting – Conversión de Tipo

Data type of a	To data type	Conversion function/type casting
unsigned, signed	std_logic_vector	std_logic_vector(a)
signed, std_logic_vector	unsigned	unsigned(a)
unsigned, std_logic_vector	signed	signed(a)
unsigned, signed	integer	to_integer(a)
natural	unsigned	to_unsigned(a, size)
integer	signed	to_signed(a, size)



Ruteo

- Hasta ahora, sabemos trabajar con algunos tipos sintetizables y realizar operaciones lógicas, aritméticas y relacionales.
- ¿Cómo seleccionar de diferentes posibles resultados, similar a las sentencias **if** y **case**?
- En hardware, esto se hace evaluando todos los posibles resultados en paralelo y *ruteando* el seleccionado a la salida. Esta selección se implementa con multiplexores.
- Vamos a ver dos maneras de escribir en VHDL sentencias concurrentes que generan dos modos distintos de ruteo: *ruteo con prioridad* y *selección de señal*.

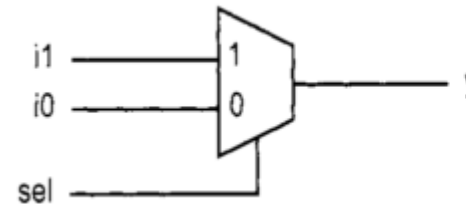
Ruteo con prioridad: *when-else*

■ Sintaxis

```
signal_name <= value_expr_1 when boolean_expr_1 else  
               value_expr_2 when boolean_expr_2 else  
               . . .  
               value_expr_n;
```

- Implica una prioridad en la asignación.
- Parecido al *if*.

■ Multiplexor



sel	y
0 (false)	i0
1 (true)	i1

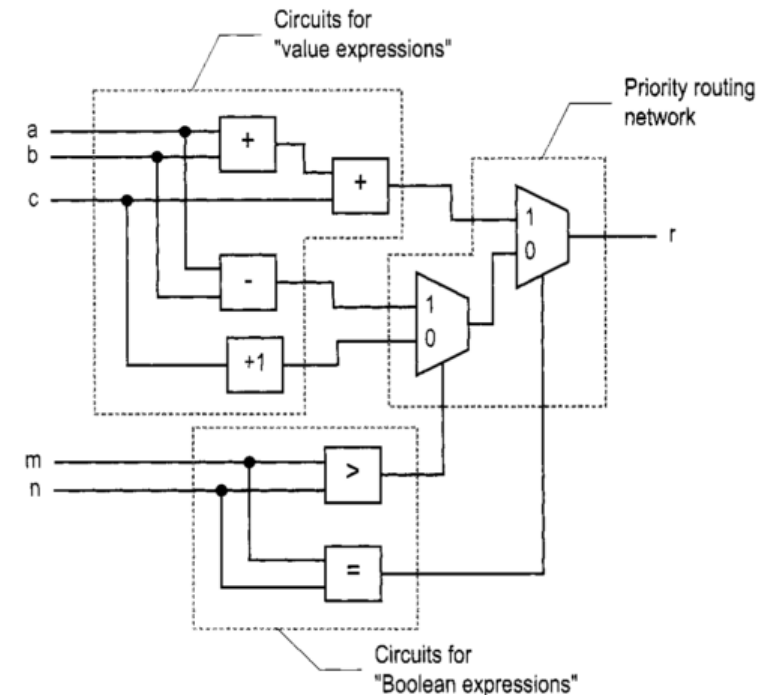
Ruteo con prioridad: Ejemplo

- Expresiones de valor y booleanas se evalúan **concurrentemente**.
- Las expresiones booleanas determinan las señales de selección de los multiplexores.
- Cada cláusula **when-else** introduce un *Mux2-1*: cuántas mas cláusulas, más larga es la cadena y por lo tanto mayores demoras.

■ Sintaxis

```
r <= a + b + c when m = n else  
    a - b      when m > n else  
    c + 1;
```

■ Ejemplo



Selección de Señal: *with-select*

- Parecido al *case*.
- Una opción tiene que ser un valor posible de **with-select**.
- Tienen que estar todos los valores y ser mutuamente excluyentes, **others**.
- En general son *std_logic_vector* o tipos enumerados.

■ Sintaxis

```
with sel select
    sig <= value_expr_1 when choice_1,
           value_expr_2 when choice_2,
           value_expr_3 when choice_3,
           . . .
           value_expr_n when others;
```

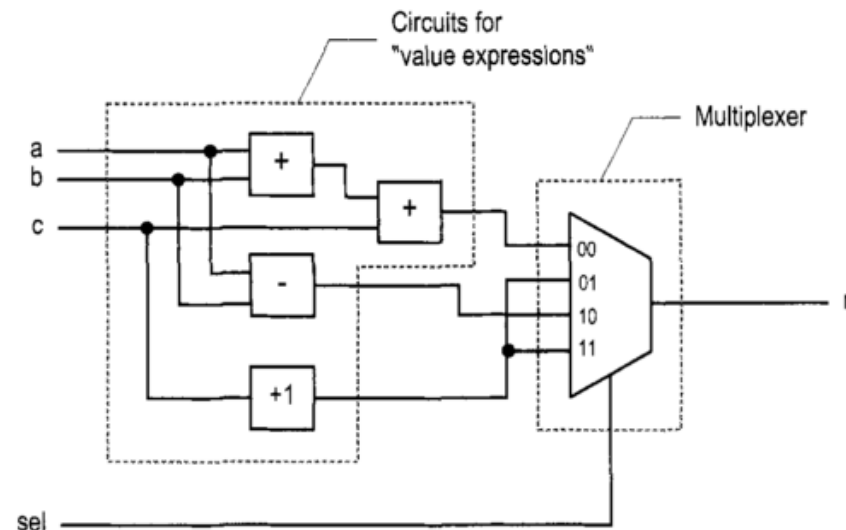
Selección de Señal: Ejemplo

- Infiere un *multiplexor 4:1*, con **sel** como señal de selección.
- El tamaño del multiplexor crece geométricamente con la cantidad de bits de **sel**.
- Nuevamente, las expresiones de valor se evalúan concurrentemente.

■ Sintaxis

```
signal sel: std_logic_vector(1 downto 0);  
...  
with sel select  
    r <= a + b + c    when "00",  
        a - b         when "10",  
        c + 1         when others;
```

■ Ejemplo



Constantes (1/2)

- Para límites de arreglos o expresiones generales.
- Permite una mejor lectura del código.

■ Sintaxis

```
constant const_name : data_type := value_expression;
```

■ Ejemplo: Comparador con un Patrón determinado

```
7 library IEEE;
8 use IEEE.STD_LOGIC_1164.ALL;
9
10 entity comp is
11     port(
12         entrada:    in std_logic_vector(3 downto 0);
13         comparador: out std_logic_vector(3 downto 0);
14     );
15 end comp;
16
17 architecture Behavioral of comp is
18     constant PATRON : std_logic_vector(3 downto 0):="1101";
19 begin
20     -- Si la entrada es igual a PATRON,
21     -- se pone en alto cada bit de comparador
22     comparador <= entrada xnor PATRON;
23
24 end Behavioral;
```

Constantes (2/2)

■ Testbench Ejemplo

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 ENTITY comp_test IS
5 END comp_test;
6
7 architecture Behavioral of comp_test is
8
9     constant DEMORA : time:=1 ns;
10
11     -- Declaro componente bajo prueba
12     component comp is
13     port(
14         entrada:    in std_logic_vector(3 downto 0);
15         comparador: out std_logic_vector(3 downto 0)
16     );
17     end component;
18
19     -- Entrada
20     signal entrada : std_logic_vector(3 downto 0);
21
22     -- Salida
23     signal comparador : std_logic_vector(3 downto 0);
24
25 begin
```

```
25 begin
26
27     -- Instancio unidad bajo prueba (uut)
28     uut: comp port map (
29         entrada => entrada,
30         comparador => comparador
31     );
32
33     -- Genero vector de pruebas
34     process
35     begin
36         -- caso 0
37         entrada <= "1100";
38         wait for DEMORA;
39         assert comparador /= "1111" report "Fallo caso 0" severity failure;
40
41         -- caso 1
42         entrada <= "1101";
43         wait for DEMORA;
44         assert comparador = "1111" report "Fallo caso 1" severity failure;
45
46         -- si llegué hasta acá, esta OK
47         assert false report "Simulacion OK!" severity note;
48         wait;
49     end process;
50
51 end;
```

Genéricos

- Con las constantes, es más fácil de leer, pero si quiero instanciar un sumador de 5 bits, el de 4 bits no me sirve.
- Los **generic** son parámetros de los módulos:
 - permiten instanciar módulos con la misma estructura, pero distintos tamaños.

■ Sintaxis

```
entity entity_name is
    generic (
        generic_name: data_type := default_values;
        generic_name: data_type := default_values;
        . . .
        generic_name: data_type := default_values
    )
    port (
        port_name: mode data_type;
        . . .
    );
end entity_name;
```

- Cargar un valor a N bits de un vector sin conocer el valor de N

```
signal: A std_logic_vector(N-1 downto 0);
-- A = "...00000..."
A <= (others => '0');
```

Genéricos: Ejemplo (1/2)

- **Ejemplo:** Conversor de Signo y Magnitud a Complemento a 2

```
7 library IEEE;
8 use IEEE.STD_LOGIC_1164.ALL;
9 use IEEE.NUMERIC_STD.ALL;
10
11 entity ConvC2 is
12     generic(N : integer := 4);
13     port(
14         x: in  std_logic_vector(N-1 downto 0);
15         y: out signed(N-1 downto 0)
16     );
17 end ConvC2;
18
19 architecture Behavioral of ConvC2 is
20 begin
21
22     -- Convierte de Signo-Magnitud a Complemento a 2
23     -- de acuerdo al bit de signo.
24     y <= signed(x) when x(N-1) = '0' else
25         not (signed('0' & x(N-2 downto 0))) + 1;
26 end Behavioral;
```

Genéricos: Ejemplo (2/2)

■ Testbench Ejemplo

```
7 library IEEE;
8 use IEEE.std_logic_1164.all;
9 use IEEE.numeric_std.all;
10
11 entity ConvC2_test is
12 end ConvC2_test;
13
14 architecture Behavioral of ConvC2_test is
15
16     -- Testbench para 3 bit
17     constant M : integer := 3;
18
19     -- Declaro componente bajo prueba
20     component ConvC2 is
21     generic(N : integer := 4);
22     port(
23         x: in  std_logic_vector(N-1 downto 0);
24         y: out signed(N-1 downto 0)
25     );
26     end component;
27
28     -- Entrada
29     signal x : std_logic_vector(M-1 downto 0);
30
31     -- Salida
32     signal y : signed(M-1 downto 0);
```

```
34 begin
35
36     -- Instancio unidad bajo prueba (uut)
37     uut: ConvC2
38     generic map (N => M)
39     port map (
40         x => x,
41         y => y
42     );
43
44     -- Genero vector de pruebas
45     process
46     begin
47         -- caso 3
48         x <= "011";
49         wait for 1 ns;
50         assert y = "011" report "Fallo caso 3" severity failure;
51
52         -- caso 2
53         x <= "010";
54         wait for 1 ns;
55         assert y = "010" report "Fallo caso 2" severity failure;
56
57         :
58     end process;
```



Fin: Funciones Lógicas Combinacionales en VHDL